MODULE 2 · WEEK 2

Pipelines & Hyperparameter Tuning

Bundle preprocessing with your model, spot-check many algorithms, and tune them properly with GridSearchCV.

Lectures 09–10 · 12–13 · 19

This week

1. **Pipelines** — preprocessing + model as one leak-proof object
2. **Spot-checking** — quickly compare many models on the same data
3. **Ensembles** — combine models for a stronger result
4. **GridSearchCV** — search hyperparameters with cross-validation
5. **Advanced pipelines** — preprocessing and tuning at the same time

Why pipelines

- A **Pipeline** chains preprocessing and the model into a single object.
- It prevents the classic leak: scaling test data with the training set's statistics.
- Bonus: your code is far easier to hand off and reuse at work.

```
from sklearn.pipeline import Pipeline
pipe = Pipeline([('scaler', MinMaxScaler()),
                 ('clf', RandomForestClassifier())])
pipe.fit(X_train, y_train)
```

Key idea A pipeline guarantees the test set never influences preprocessing — no accidental leakage.

Spot-checking models

- Before tuning, **spot-check**: run several algorithms with defaults on the same CV split.
- See which model families are even in the running before investing in tuning.
- Compare them fairly with the same cross-validation and metric.

```
for name, model in models.items():
    scores = cross_val_score(model, X, y, cv=cv, scoring='f1')
    print(name, scores.mean())
```

Ensemble methods

- Combine several models so their errors cancel out and the strong signals reinforce.
- **Bagging** (Random Forest) reduces variance; **boosting** reduces bias.
- **Voting / stacking** blends different model families into one prediction.

Hyperparameter tuning with GridSearchCV

- Hyperparameters are the knobs you set *before* training (tree depth, n_estimators, learning rate).
- **GridSearchCV** tries every combination in a grid, scored by cross-validation.
- It returns the best settings — and you read the results table to understand the trade-offs.

```
from sklearn.model_selection import GridSearchCV
grid = {'clf__n_estimators': [100, 300],
        'clf__max_depth': [5, 10, None]}
gs = GridSearchCV(pipe, grid, cv=5, scoring='f1')
gs.fit(X_train, y_train); gs.best_params_
```

Key idea Tuning squeezes out the last few points of performance — and in fraud or medicine, those points matter.

Advanced pipelines: preprocess + tune together

- Put preprocessing *and* the model in one pipeline, then grid-search across both.
- Use the `step__param` naming so the grid can reach inside the pipeline.
- Now every fold scales, resamples, fits, and tunes — all leak-free, in one object.

```
grid = {'scaler': [MinMaxScaler(), StandardScaler()],
        'clf__max_depth': [5, 10, None]}
```

Lab this week

- Build a sklearn Pipeline that chains preprocessing and a classifier.
- Run GridSearchCV across multiple models and hyperparameter grids, compare fairly with CV, and read the results table to pick a winner.

In the labs repo: `Module2/Week2_Tuning/grid_search_classification.py`

Key takeaways

- Pipelines bundle preprocessing + model into one leak-proof, reusable object.
- Spot-check many models first; only tune the ones worth tuning.
- GridSearchCV searches hyperparameters with cross-validation and returns the best settings.
- Advanced pipelines tune preprocessing and the model together — fairly and safely.